Object-Oriented Programming in Java:

Object-Oriented Programming (OOP) in Java is a paradigm that models real-world entities using classes and objects, enabling modular, reusable, and maintainable code. The four core principles are encapsulation, inheritance, polymorphism, and abstraction, each of which helps structure complex applications efficiently.

🔑 Core Concepts of OOP in Java

1. Class & Object

- **Class**: A blueprint that defines properties (fields) and behaviors (methods).
- **Object**: An instance of a class representing a real-world entity.
- **Example**:

```
class Car {
   String brand;
   int speed;
   void drive() {
      System.out.println("Car is driving...");
   }
}
Car myCar = new Car(); // Object creation
```

2. Encapsulation

- **Definition**: Wrapping data (fields) and methods into a single unit (class).

- **Benefit**: Protects internal state by restricting direct access.
- **Implementation**: Use private fields with public getters/setters.
- **Example**:

```
class Student {
   private String name;
   public String getName() { return name; }
   public void setName(String name) { this.name = name; }
}
```

3. Inheritance

- **Definition**: Mechanism where one class derives properties and behaviors from another.
- **Keyword**: extends
- **Benefit**: Promotes code reuse.
- **Example**:

```
class Animal {
   void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
   void sound() { System.out.println("Bark"); }
}
```

4. Polymorphism

- **Definition**: Ability of an object to take many forms.
- **Types**:

    - **Compile-time (Method Overloading)**: Same method name, different parameters.
    - **Runtime (Method Overriding)**: Subclass provides specific implementation.

- **Example**:

```
class Calculator {
   int add(int a, int b) { return a + b; }
   double add(double a, double b) { return a + b; }
}
```

5. Abstraction

- **Definition**: Hiding implementation details and exposing only essential features.
- **Tools**: Abstract classes & interfaces.
- **Example**:

```
interface Vehicle {
   void start();
}
class Bike implements Vehicle {
   public void start() { System.out.println("Bike started"); }
}
```

📊 **Quick Comparison of OOP Principles**

Principle	Keyword(s)	Purpose	Example
Encapsulation	private, public	Data hiding & controlled access	Getters/Setters
Inheritance	extends	Code reuse	Dog extends Animal
Polymorphism	Overloading/Overriding	Flexibility in behavior	Multiple add() methods
Abstraction	abstract, interface	Hide complexity	Vehicle interface

⚠️ **Key Considerations**

- **Encapsulation** ensures security but requires careful design of access methods.
- **Inheritance** can lead to tight coupling if overused; prefer composition when possible.
- **Polymorphism** makes code flexible but can complicate debugging.
- **Abstraction** improves maintainability but may increase initial design complexity.

Understand OOP foundations: classes, objects, constructors, encapsulation, and inheritance:

1. Classes:

- **Definition**: A class is a blueprint for creating objects. It defines attributes (fields) and behaviors (methods).
- **Example**:

```
class Car {
    String brand;
    int speed;
    void drive() {
        System.out.println("Car is driving...");
    }
}
```

◇ 2. Objects

- **Definition**: An object is an instance of a class. It represents a real-world entity.
- **Example**:

```
public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // Object creation
        myCar.brand = "Toyota";
        myCar.speed = 120;
        myCar.drive(); // Output: Car is driving...
    }
}
```

◇ 3. Constructors

- **Definition**: A constructor is a special method used to initialize objects. It has the same name as the class and no return type.
- **Example**:

```
class Student {
    String name;
    int age;
    // Constructor
    Student(String n, int a) {
        name = n;
        age = a;
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Alice", 20);
        System.out.println(s1.name + " - " + s1.age);
    }
}
```

◇ 4. Encapsulation

- **Definition**: Encapsulation is the practice of hiding data by making fields private and providing controlled access through getters and setters.
- **Example**:

```
class BankAccount {
    private double balance;
    public double getBalance() {
        return balance;
    }
    public void deposit(double amount) {
        balance += amount;
    }
}
```

◇ 5. Inheritance

- **Definition**: Inheritance allows one class to acquire properties and behaviors of another. It promotes code reuse.
- **Keyword**: extends
- **Example**:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound(); // Output: Dog barks
    }
```

Master polymorphism, abstraction, interface implementation, method overriding, and access modifiers:

1. Polymorphism:

- **Definition**: The ability of an object to take many forms. It allows methods to behave differently based on context.
- **Types**:
  - **Compile-time (Overloading)** → Same method name, different parameter lists.
  - **Runtime (Overriding)** → Subclass provides its own implementation of a method.
- **Example**:

```
class Shape {
    void draw() { System.out.println("Drawing a shape"); }
```

```
}

class Circle extends Shape {

   void draw() { System.out.println("Drawing a circle"); }

}

public class Main {

   public static void main(String[] args) {

      Shape s = new Circle(); // Runtime polymorphism

      s.draw(); // Output: Drawing a circle

   }

}
```

2. Abstraction:

- **Definition**: Hiding implementation details and exposing only essential features.
- **Tools**: Abstract classes and interfaces.
- **Example (Abstract Class)**:

```
abstract class Animal {

   abstract void sound(); // abstract method

}

class Dog extends Animal {

   void sound() { System.out.println("Bark"); }

}
```

3. Interface Implementation:

- **Definition**: An interface defines a contract (methods without implementation). Classes must implement them.
- **Example**:

```
interface Vehicle {
    void start();
}
class Bike implements Vehicle {
    public void start() { System.out.println("Bike started"); }
}
```

4. Method Overriding:

- **Definition**: When a subclass provides a specific implementation of a method already defined in its parent class.
- **Rules**:
  - Must have the same method name and parameters.
  - Must have compatible return type.
  - Cannot override final methods.
- **Example**:

```
class Parent {
    void greet() { System.out.println("Hello from Parent"); }
}
class Child extends Parent {
    @Override
    void greet() { System.out.println("Hello from Child"); }
}
```

5. Access Modifiers:

- **Definition**: Keywords that control visibility of classes, methods, and variables.
- **Types**:
  - public → Accessible everywhere.
  - protected → Accessible within package + subclasses.
  - default (no modifier) → Accessible only within package.
  - private → Accessible only within the class.
- **Example**:

```
class Demo {

   public int x = 10;       // Accessible everywhere

   protected int y = 20;    // Accessible in package + subclasses

   int z = 30;              // Default (package-private)

   private int w = 40;      // Accessible only in this class

}
```

☑ **Quick Recap:**

- **Polymorphism** → Many forms (overloading & overriding).
- **Abstraction** → Hide details, show essentials.
- **Interface** → Contract that classes must fulfill.
- **Method Overriding** → Subclass customizes parent behavior.
- **Access Modifiers** → Control visibility and security.

Build a Java Banking Management System: deposit, withdraw, balance check, and transaction history:

Definition:

A **Banking Management System** in Java is an application that simulates basic banking operations using **OOP principles**:

- **Encapsulation** → Secure account data with private fields.
- **Inheritance/Polymorphism** → Extend functionality for different account types.
- **Abstraction** → Hide internal details, expose only essential operations.
- **Transaction History** → Store records of deposits and withdrawals.

💻 **Example Code:**

```
import java.util.ArrayList;

import java.util.List;

class BankAccount {

   private String accountHolder;

   private double balance;

   private List<String> transactionHistory;

   // Constructor

   public BankAccount(String accountHolder, double initialBalance) {

      this.accountHolder = accountHolder;

      this.balance = initialBalance;

      this.transactionHistory = new ArrayList<>();

      transactionHistory.add("Account created with balance: " +
initialBalance);

   }

   // Deposit method

   public void deposit(double amount) {
```

```
    if (amount > 0) {
        balance += amount;
        transactionHistory.add("Deposited: " + amount);
        System.out.println("Deposit successful!");
    } else {
        System.out.println("Invalid deposit amount.");
    }
}
// Withdraw method
public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
        transactionHistory.add("Withdrew: " + amount);
        System.out.println("Withdrawal successful!");
    } else {
        System.out.println("Insufficient balance or invalid amount.");
    }
}
// Balance check
public double getBalance() {
    return balance;
```

```
    }

    // Transaction history

    public void printTransactionHistory() {

        System.out.println("Transaction History for " + accountHolder +
":");

        for (String record : transactionHistory) {

            System.out.println(record);

        }

    }

}

// Main class to test

public class BankingSystem {

    public static void main(String[] args) {

        BankAccount account = new BankAccount("Alice", 1000);

        account.deposit(500);

        account.withdraw(200);

        System.out.println("Current Balance: " + account.getBalance());

        account.printTransactionHistory();

    }

}
```

📝 **Output Example**

Code

Deposit successful!

Withdrawal successful!

Current Balance: 1300.0

Transaction History for Alice:

Account created with balance: 1000.0

Deposited: 500.0

Withdrew: 200.0

Key Takeaways:

- **Encapsulation**: Balance and history are private, accessed via methods.
- **Constructor**: Initializes account with a starting balance.
- **Transaction History**: Stored in a List<String> for easy tracking.
- **Flexibility**: You can extend this system to support multiple accounts, interest calculation, or online transfers.

Establish class structures, design class diagrams, and prepare design documentations:

Definition:

A **Banking Management System** in Java is an application that simulates basic banking operations using **Object-Oriented Programming (OOP)** principles:

- **Encapsulation** → Secure account data with private fields.
- **Constructors** → Initialize accounts with starting balances.
- **Methods** → Provide controlled operations like deposit, withdraw, and balance check.

- **Collections** → Store transaction history for tracking.

💻 Example Code

```
import java.util.ArrayList;

import java.util.List;

class BankAccount {

    private String accountHolder;

    private double balance;

    private List<String> transactionHistory;

    // Constructor

    public BankAccount(String accountHolder, double initialBalance) {

        this.accountHolder = accountHolder;

        this.balance = initialBalance;

        this.transactionHistory = new ArrayList<>();

        transactionHistory.add("Account created with balance: " +
initialBalance);

    }

    // Deposit method

    public void deposit(double amount) {

        if (amount > 0) {

            balance += amount;

            transactionHistory.add("Deposited: " + amount);
```

```
            System.out.println("Deposit successful!");
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }
    // Withdraw method
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            transactionHistory.add("Withdrew: " + amount);
            System.out.println("Withdrawal successful!");
        } else {
            System.out.println("Insufficient balance or invalid amount.");
        }
    }
    // Balance check
    public double getBalance() {
        return balance;
    }
    // Transaction history
    public void printTransactionHistory() {
```

```
        System.out.println("Transaction History for " + accountHolder +
":");
        for (String record : transactionHistory) {
            System.out.println(record);
        }
    }
}
// Main class to test
public class BankingSystem {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("Alice", 1000);
        account.deposit(500);
        account.withdraw(200);
        System.out.println("Current Balance: " + account.getBalance());


        account.printTransactionHistory();
    }
}
```

Sample Output

Code

Deposit successful!

Withdrawal successful!

Current Balance: 1300.0

Transaction History for Alice:

Account created with balance: 1000.0

Deposited: 500.0

Withdrew: 200.0

☑ **Key Takeaways:**

- **Encapsulation** secures balance and transaction history.
- **Constructor** initializes account with a starting balance.
- **Methods** provide controlled operations (deposit, withdraw, balance check).
- **Transaction History** is stored in a list for easy tracking.